

Лабораторная работа 3: Логика хранения данных и работа со списками

Введение в паттерн Repository и работу с данными

Зачем нужен Repository?

В предыдущей работе мы создали приложение с формой редактирования, но данные не сохранялись — каждый раз мы работали с одним объектом. Теперь нам нужно:

- **Хранить множество объектов** (список резерваций, заказов, книг и т.д.)
- **Выполнять операции CRUD** (Create, Read, Update, Delete)
- **Отделить логику хранения от UI** — чтобы в будущем можно было легко заменить хранение в памяти на базу данных

Repository (Репозиторий) — это паттерн проектирования, который инкапсулирует логику доступа к данным. Repository предоставляет единообразный интерфейс для работы с коллекцией объектов.

Преимущества паттерна Repository

1. **Разделение ответственности** — UI не знает, где хранятся данные (в памяти, в файле, в базе данных)
2. **Тестируемость** — можно легко создать мок-версию репозитория для тестирования UI
3. **Гибкость** — можно заменить in-memory хранилище на реальную базу данных без изменения UI
4. **Единообразие** — все операции с данными выполняются через один интерфейс

Что такое CRUD?

CRUD — это акроним, обозначающий четыре базовые операции с данными:

- **Create** — создание нового объекта
- **Read** — чтение объектов (получение одного или всех)
- **Update** — обновление существующего объекта
- **Delete** — удаление объекта

Создание проектов для работы с данными

Шаг 1: Создание проекта Data.Interfaces

Проект `Data.Interfaces` будет содержать **интерфейсы** репозитория — контракты, описывающие, какие операции можно выполнять с данными.

Зачем нужен отдельный проект для интерфейсов?

- UI будет зависеть только от интерфейсов, а не от конкретной реализации
- Можно будет создать несколько реализаций (in-memory, файл, база данных) без изменения UI
- Соблюдается принцип **Dependency Inversion** из SOLID

Действия:

1. В Visual Studio, ПКМ по Solution → Add → New Project
2. Выберите тип проекта **Class Library** (.NET 9.0)

3. Назовите проект: `<ИмяПроекта>.Data.Interfaces` (например, `BeautyDB.Data.Interfaces`)
4. Удалите автоматически созданный файл `Class1.cs`

Шаг 2: Создание проекта `Data.InMemory`

Проект `Data.InMemory` будет содержать **реализацию** репозиториев, которая хранит данные в памяти приложения (в `List<T>`).

Действия:

1. ПКМ по Solution → Add → New Project
2. Выберите тип проекта **Class Library** (.NET 9.0)
3. Назовите проект: `<ИмяПроекта>.Data.InMemory` (например, `BeautyDB.Data.InMemory`)
4. Удалите автоматически созданный файл `Class1.cs`

Шаг 3: Добавление ссылок между проектами

Теперь нужно настроить зависимости между проектами. Важно настраивать их в правильном порядке — от базовых к зависимым:

`Data.Interfaces` зависит от `Domain`:

1. ПКМ по проекту `<ИмяПроекта>.Data.Interfaces` → Add → Project Reference
2. Выберите `<ИмяПроекта>.Domain`
3. Нажмите OK

`Data.InMemory` зависит от `Data.Interfaces` и `Domain`:

1. ПКМ по проекту `<ИмяПроекта>.Data.InMemory` → Add → Project Reference
2. Выберите `<ИмяПроекта>.Data.Interfaces` и `<ИмяПроекта>.Domain`
3. Нажмите OK

`UI` зависит от `Data.Interfaces` и `Data.InMemory`:

1. ПКМ по проекту `<ИмяПроекта>.UI` → Add → Project Reference
2. Выберите `<ИмяПроекта>.Data.Interfaces` и `<ИмяПроекта>.Data.InMemory`
3. Нажмите OK

Схема зависимостей (от базовых к зависимым):

Domain	(базовый слой, не зависит ни от чего)
↑	
Data.Interfaces	(зависит от Domain)
↑	
Data.InMemory → Domain	(зависит от Data.Interfaces и Domain)
↑	
UI → Data.Interfaces	(зависит от Data.Interfaces и Data.InMemory)

Создание интерфейсов репозиториев

Что такое интерфейс?

Интерфейс в C# — это контракт, который описывает, какие методы должен реализовать класс. Интерфейс содержит только сигнатуры методов (без реализации).

Пример:

```

public interface IAnimal
{
    void MakeSound(); // Объявление метода без реализации
}

public class Dog : IAnimal
{
    public void MakeSound() // Реализация метода
    {
        Console.WriteLine("Woof!");
    }
}

```

Создание интерфейса репозитория для основной сущности

Примечание: Замените `Reservation` на вашу основную сущность (заказ, книга, заявка и т.д.), для которой будет создан список.

Файл: `<ИмяПроекта>.Data.Interfaces/I<Сущность>Repository.cs` (например, `IReservationRepository.cs`)

```

namespace <ИмяПроекта>.Data.Interfaces;

using <ИмяПроекта>.Domain;

public interface I<Сущность>Repository
{
    List<<Сущность>> GetAll();
    Guid Add(<Сущность> item);
    void Update(<Сущность> item);
    void Delete(<Сущность> item);
}

```

Конкретный пример для резервации салона красоты:

Файл: `BeautyDB.Data.Interfaces/IReservationRepository.cs`

```

namespace BeautyDB.Data.Interfaces;

using BeautyDB.Domain;

public interface IReservationRepository
{
    List<Reservation> GetAll();
    Guid Add(Reservation reservation);
    void Update(Reservation reservation);
    void Delete(Reservation reservation);
}

```

Разбор кода:

- `List<Reservation> GetAll()` — возвращает список всех резерваций
- `Guid Add(Reservation reservation)` — добавляет новую резервацию и возвращает её Id
- `void Update(Reservation reservation)` — обновляет существующую резервацию
- `void Delete(Reservation reservation)` — удаляет резервацию

Важно: Интерфейс не содержит реализации — это только контракт.

Создание интерфейса для связанной сущности

Если в вашей предметной области есть связанные сущности (например, мастера для резерваций, авторы для книг, категории для товаров), создайте для них отдельные репозитории.

Пример для мастеров:

Файл: `BeautyDB.Data.Interfaces/IMasterRepository.cs`

```
namespace BeautyDB.Data.Interfaces;

using BeautyDB.Domain;

public interface IMasterRepository
{
    List<Master> GetAll();
}
```

Примечание: Для справочных сущностей (мастера, категории, авторы) часто достаточно только метода `GetAll()` до тех пор, пока они не изменяются при работе программы.

Реализация in-memory репозиториев

Что такое in-memory хранилище?

In-memory (в памяти) означает, что данные хранятся в оперативной памяти приложения (в коллекциях типа `List<T>`, `Dictionary<K,V>` и т.д.).

Особенности:

- **Быстро** — доступ к данным в памяти мгновенный
- **Временно** — данные теряются при закрытии приложения
- **Просто** — не требуется настройка базы данных

Реализация репозитория для основной сущности

Файл: `<ИмяПроекта>.Data.InMemory/<Сущность>Repository.cs` (например, `ReservationRepository.cs`)

```
namespace <ИмяПроекта>.Data.InMemory;

using <ИмяПроекта>.Data.Interfaces;
using <ИмяПроекта>.Domain;

public class <Сущность>Repository : I<Сущность>Repository
{
    private readonly List<<Сущность>> _items = new();

    public List<<Сущность>> GetAll()
    {
        return _items.Select(item => item.Clone()).ToList();
    }

    public Guid Add(<Сущность> item)
    {
        var clone = item.Clone();
        clone.Id = Guid.NewGuid();
        _items.Add(clone);
        return clone.Id;
    }
}
```

```

public void Update(<Сущность> item)
{
    var index = _items.FindIndex(x => x.Id == item.Id);
    if (index >= 0)
    {
        _items[index] = item.Clone();
    }
}

public void Delete(<Сущность> item)
{
    _items.RemoveAll(x => x.Id == item.Id);
}
}

```

Конкретный пример для резервации:

Файл: BeautyDB.Data.InMemory/ReservationRepository.cs

```

namespace BeautyDB.Data.InMemory;

using BeautyDB.Data.Interfaces;
using BeautyDB.Domain;

public class ReservationRepository : IReservationRepository
{
    private readonly List<Reservation> _reservations = new();

    public List<Reservation> GetAll()
    {
        return _reservations.Select(r => r.Clone()).ToList();
    }

    public Guid Add(Reservation reservation)
    {
        var clone = reservation.Clone();
        clone.Id = Guid.NewGuid();
        _reservations.Add(clone);
        return clone.Id;
    }

    public void Update(Reservation reservation)
    {
        var index = _reservations.FindIndex(r => r.Id == reservation.Id);
        if (index >= 0)
        {
            _reservations[index] = reservation.Clone();
        }
    }

    public void Delete(Reservation reservation)
    {
        _reservations.RemoveAll(r => r.Id == reservation.Id);
    }
}

```

Разбор кода:

Хранилище данных

```
private readonly List<Reservation> _reservations = new();
```

Репозиторий хранит все объекты в приватном списке. Ключевое слово `readonly` означает, что ссылку на список нельзя изменить после создания (но содержимое списка можно добавлять/удалять).

Операция Create — метод Add()

```
public Guid Add(Reservation reservation)
{
    var clone = reservation.Clone();
    clone.Id = Guid.NewGuid();
    _reservations.Add(clone);
    return clone.Id;
}
```

Что делает:

1. Генерирует новый уникальный Id для объекта (через `Guid.NewGuid()`)
2. Добавляет объект в список
3. Возвращает Id, чтобы UI знал идентификатор созданного объекта

Зачем возвращать Id?

UI может использовать этот Id для последующих операций (например, сразу открыть созданный объект для редактирования).

Операция Read — метод GetAll()

```
public List<Reservation> GetAll()
{
    return _reservations.Select(r => r.Clone()).ToList();
}
```

Что делает:

Возвращает список всех резерваций.

Используемые техники:

- `Select()` — LINQ-метод, который применяет функцию к каждому элементу коллекции
- `ToList()` — преобразует результат в `List<T>`

Примечание: Метод `Clone()` создаёт копии объектов для защиты внутреннего состояния репозитория от случайных изменений в UI. Это необязательная техника, но она делает код более безопасным.

Операция Update — метод Update()

```
public void Update(Reservation reservation)
{
    var index = _reservations.FindIndex(r => r.Id == reservation.Id);
    if (index >= 0)
    {
        _reservations[index] = reservation.Clone();
    }
}
```

Что делает:

1. Ищет объект с нужным Id в списке

2. Если нашёл — заменяет старый объект на новый
3. Если не нашёл — ничего не делает (молча игнорирует)

Используемые техники:

- `FindIndex()` — ищет элемент по условию и возвращает его индекс
- Лямбда-выражение `r => r.Id == reservation.Id` — условие поиска
- `FindIndex()` возвращает `-1`, если ничего не найдено

Операция Delete — метод Delete()

```
public void Delete(Reservation reservation)
{
    _reservations.RemoveAll(r => r.Id == reservation.Id);
}
```

Что делает:

Удаляет объект с указанным Id из списка.

Используемые техники:

- `RemoveAll()` — удаляет все элементы, соответствующие условию
- В нашем случае Id уникален, поэтому удалится максимум один элемент

Реализация репозитория для связанной сущности

Файл: `BeautyDB.Data.InMemory/MasterRepository.cs`

```
namespace BeautyDB.Data.InMemory;

using BeautyDB.Data.Interfaces;
using BeautyDB.Domain;

public class MasterRepository : IMasterRepository
{
    private readonly List<Master> _masters = new();

    public List<Master> GetAll()
    {
        return _masters.ToList();
    }

    public void Seed(List<Master> masters)
    {
        _masters.Clear();
        _masters.AddRange(masters);
    }
}
```

Разбор кода:

- `Seed()` — вспомогательный метод для заполнения репозитория тестовыми данными
- Для справочных данных (мастера, категории) `Clone()` можно не использовать до тех пор, пока они не изменяются при работе программы

О методе Clone() — опциональная техника защиты данных

В примерах кода выше используется метод `Clone()`, который создаёт **копию объекта**. Это **опциональная техника**, которая защищает репозиторий от случайных изменений.

Проблема без `Clone()`:

```
// Репозиторий возвращает прямую ссылку на объект
public List<Reservation> GetAll()
{
    return _reservations.ToList(); // ToList создаёт новый список, но объекты внутри – те же
}

// В UI можем случайно изменить данные
var reservations = repository.GetAll();
reservations[0].Client.Name = "Новое имя"; // Изменяет данные прямо в репозитории!
```

Решение с `Clone()`:

```
// Репозиторий возвращает копии объектов
public List<Reservation> GetAll()
{
    return _reservations.Select(r => r.Clone()).ToList();
}

// В UI работаем с копиями
var reservations = repository.GetAll();
reservations[0].Client.Name = "Новое имя"; // Меняет только копию, репозиторий не затронут
```

Как реализовать `Clone()`

Метод `Clone()` создаёт новый объект с теми же значениями свойств.

Пример 1 — Простой класс без вложенных объектов:

```
public class Master
{
    public Guid Id { get; set; }
    public string Name { get; set; } = "";

    public Master Clone()
    {
        return new Master
        {
            Id = this.Id,
            Name = this.Name
        };
    }
}
```

Пример 2 — Класс с вложенными объектами:

```
public class Client
{
    public string Name { get; set; } = "";
    public string Phone { get; set; } = "";

    public Client Clone()
    {
        return new Client
        {

```



```

        Name = this.Name,
        Phone = this.Phone
    };
}

}

public class Reservation
{
    public Guid Id { get; set; }
    public Client Client { get; set; } = new();
    public Master Master { get; set; } = new();
    public string ServiceDescription { get; set; } = "";
    public DateOnly Date { get; set; }
    public TimeOnly StartTime { get; set; }
    public TimeSpan Duration { get; set; }
    public ReservationStatus Status { get; set; }

    public Reservation Clone()
    {
        return new Reservation
        {
            Id = this.Id,
            Client = this.Client.Clone(), // Клонировем вложенный объект
            Master = this.Master,         // Master – справочник, просто копируем ссылку
            ServiceDescription = this.ServiceDescription,
            Date = this.Date,
            StartTime = this.StartTime,
            Duration = this.Duration,
            Status = this.Status
        };
    }
}

```

Что нужно клонировать:

- **Уникальные данные для каждой записи (Client)** → вызываем `.Clone()`
 - У каждой резервации свой клиент с уникальными именем и телефоном
 - Если изменить клиента в одной резервации, это не должно затронуть других клиентов
- **Общие справочные данные (Master)** → просто копируем ссылку
 - Один и тот же мастер используется во многих резервациях
 - Мы не изменяем свойства мастера через резервацию, только выбираем из списка
- **Типы значений (int, Guid, DateOnly, enum)** → копируются автоматически
 - Это примитивные типы, которые всегда копируются по значению
- **Строки (string)** → иммутабельны, безопасно копировать ссылку
 - Строки нельзя изменить после создания, поэтому копирование ссылки безопасно

Плюсы и минусы Clone()

Плюсы:

- Защита от случайных изменений данных в репозитории
- Явный контроль над тем, что именно копируется
- Проще отлаживать проблемы с данными

Минусы:

- Нужно реализовывать метод для каждого класса
- Небольшие затраты памяти и времени на создание копий
- При добавлении нового свойства нужно обновить `Clone()`

Работа без Clone()

Вы можете убрать Clone() из всего кода репозитория:

```
public List<Reservation> GetAll()
{
    return _reservations.ToList(); // Без Clone()
}

public Guid Add(Reservation reservation)
{
    reservation.Id = Guid.NewGuid();
    _reservations.Add(reservation); // Без Clone()
    return reservation.Id;
}

public void Update(Reservation reservation)
{
    var index = _reservations.FindIndex(r => r.Id == reservation.Id);
    if (index >= 0)
    {
        _reservations[index] = reservation; // Без Clone()
    }
}
```

Введение в DataGrid

Что такое DataGrid?

DataGrid — это элемент управления WPF для отображения табличных данных. Он похож на таблицу Excel: имеет строки, столбцы, заголовки и поддерживает прокрутку.

Основные возможности DataGrid:

- Автоматическое создание столбцов на основе свойств объектов
- Ручная настройка столбцов (ширина, заголовки, форматирование)
- Выбор строк (одиночный или множественный)
- Сортировка по столбцам (по умолчанию)
- Редактирование ячеек (можно отключить через IsReadOnly="True")

Базовый пример DataGrid

```
<DataGrid ItemsSource="{Binding Reservations}"
    SelectedItem="{Binding SelectedReservation}"
    AutoGenerateColumns="False"
    IsReadOnly="True"
    SelectionMode="Single">
    <DataGrid.Columns>
        <DataGridTextColumn Header="Имя" Binding="{Binding Client.Name}" Width="150"/>
        <DataGridTextColumn Header="Телефон" Binding="{Binding Client.Phone}" Width="120"/>
        <DataGridTextColumn Header="Дата" Binding="{Binding Date}" Width="100"/>
    </DataGrid.Columns>
</DataGrid>
```

Разбор свойств:

- ItemsSource="{Binding Reservations}" — источник данных (коллекция объектов)

- `SelectedItem="{Binding SelectedReservation}"` — выбранная строка привязывается к свойству
- `AutoGenerateColumns="False"` — отключает автоматическое создание столбцов (мы сами их настроим)
- `IsReadOnly="True"` — запрещает редактирование ячеек
- `SelectionMode="Single"` — можно выбрать только одну строку

Типы столбцов DataGrid

DataGridTextColumn — основной тип столбца для простых данных:

```
<DataGridTextColumn Header="Имя" Binding="{Binding Name}" Width="150"/>
```

DataGridTemplateColumn — для данных, требующих преобразования (например, через конвертер):

```
<DataGridTemplateColumn Header="Статус" Width="120">
  <DataGridTemplateColumn.CellTemplate>
    <DataTemplate>
      <TextBlock Text="{Binding Status, Converter={StaticResource StatusConverter}}"/>
    </DataTemplate>
  </DataGridTemplateColumn.CellTemplate>
</DataGridTemplateColumn>
```

Важно: Не забудьте объявить конвертер в `Window.Resources`.

Настройка ширины столбцов

- **Фиксированная ширина:** `Width="150"`
- **Авто-ширина:** `Width="Auto"` (подстраивается под содержимое)
- **Оставшееся пространство:** `Width="*"` (занимает всё свободное место)

Пример:

```
<DataGrid.Columns>
  <DataGridTextColumn Header="ID" Binding="{Binding Id}" Width="80"/>
  <DataGridTextColumn Header="Имя" Binding="{Binding Name}" Width="150"/>
  <DataGridTextColumn Header="Описание" Binding="{Binding Description}" Width="*" />
</DataGrid.Columns>
```

Создание формы списка

Шаг 1: Создание окна ListWindow

Действия:

1. ПКМ по проекту `<ИмяПроекта>.UI` → Add → Window (WPF)
2. Назовите окно: `<Сущность>sListWindow.xaml` (например, `ReservationsListWindow.xaml`)

Шаг 2: Разметка XAML для формы списка

Файл: `<ИмяПроекта>.UI/<Сущность>sListWindow.xaml` (например, `ReservationsListWindow.xaml`)

```
<Window x:Class="ИмяПроекта.UI.<Сущность>sListWindow"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="clr-namespace:ИмяПроекта.UI"
  Title="Список записей" Height="600" Width="1000">
```

```

<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>

    <!-- Заголовок -->
    <TextBlock Grid.Row="0" Text="Список записей" FontSize="24" FontWeight="Bold"
Margin="10" />

    <!-- Панель с кнопками -->
    <StackPanel Grid.Row="1" Orientation="Horizontal" Margin="10">
        <Button Content="Создать" Width="100" Height="30" Margin="0,0,10,0"
Click="CreateButton_Click" />
        <Button Content="Редактировать" Width="120" Height="30" Margin="0,0,10,0"
Click="EditButton_Click" />
        <Button Content="Удалить" Width="100" Height="30" Click="DeleteButton_Click" />
    </StackPanel>

    <!-- Таблица с данными -->
    <DataGrid Grid.Row="2"
        ItemsSource="{Binding Items}"
        SelectedItem="{Binding SelectedItem}"
        AutoGenerateColumns="False"
        IsReadOnly="True"
        SelectionMode="Single"
        Margin="10">
        <DataGrid.Columns>
            <!-- Добавьте столбцы для ваших свойств -->
            <DataGridTextColumn Header="Название поля 1" Binding="{Binding Свойство1}"
Width="150" />
            <DataGridTextColumn Header="Название поля 2" Binding="{Binding Свойство2}"
Width="120" />
            <DataGridTextColumn Header="Название поля 3" Binding="{Binding Свойство3}"
Width="*" />
        </DataGrid.Columns>
    </DataGrid>
</Grid>
</Window>

```

Конкретный пример для резерваций салона красоты:

Файл: BeautyDB.UI/ReservationsListWindow.xaml

```

<Window x:Class="BeautyDB.UI.ReservationsListWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="clr-namespace:BeautyDB.UI"
    xmlns:converters="clr-namespace:BeautyDB.UI.Converters"
    Title="Список записей" Height="600" Width="1200">
    <Window.Resources>
        <converters:ReservationStatusConverter x:Key="StatusConverter" />
    </Window.Resources>
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
    </Grid>

```

```

        <TextBlock Grid.Row="0" Text="Записи клиентов" FontSize="24" FontWeight="Bold"
Margin="10"/>

        <StackPanel Grid.Row="1" Orientation="Horizontal" Margin="10">
            <Button Content="Создать" Width="100" Height="30" Margin="0,0,10,0"
Click="CreateButton_Click"/>
            <Button Content="Редактировать" Width="120" Height="30" Margin="0,0,10,0"
Click="EditButton_Click"/>
            <Button Content="Удалить" Width="100" Height="30" Click="DeleteButton_Click"/>
        </StackPanel>

        <DataGrid Grid.Row="2"
            ItemsSource="{Binding Reservations}"
            SelectedItem="{Binding SelectedReservation}"
            AutoGenerateColumns="False"
            IsReadOnly="True"
            SelectionMode="Single"
            Margin="10">
            <DataGrid.Columns>
                <DataGridTextColumn Header="Мастер" Binding="{Binding Master.Name}" Width="150"/>
                <DataGridTextColumn Header="Дата" Binding="{Binding Date}" Width="100"/>
                <DataGridTextColumn Header="Время" Binding="{Binding StartTime}" Width="80"/>
                <DataGridTextColumn Header="Длительность" Binding="{Binding Duration}"
Width="100"/>
                <DataGridTextColumn Header="Клиент" Binding="{Binding Client.Name}" Width="150"/>
                <DataGridTextColumn Header="Телефон" Binding="{Binding Client.Phone}"
Width="120"/>
                <DataGridTextColumn Header="Услуга" Binding="{Binding ServiceDescription}"
Width="*" />
                <DataGridTemplateColumn Header="Статус" Width="120">
                    <DataGridTemplateColumn.CellTemplate>
                        <DataTemplate>
                            <TextBlock Text="{Binding Status, Converter={StaticResource
StatusConverter}}"/>
                        </DataTemplate>
                    </DataGridTemplateColumn.CellTemplate>
                </DataGridTemplateColumn>
            </DataGrid.Columns>
        </DataGrid>
    </Grid>
</Window>

```

Разбор структуры окна:

1. Grid с тремя строками:

- Строка 0 (Height="Auto") — заголовок окна
- Строка 1 (Height="Auto") — панель с кнопками действий
- Строка 2 (Height="*") — таблица с данными (занимает всё оставшееся пространство)

2. TextBlock для заголовка:

```
<TextBlock Grid.Row="0" Text="Записи клиентов" FontSize="24" FontWeight="Bold" Margin="10"/>
```

- FontSize="24" — крупный шрифт для заголовка
- FontWeight="Bold" — жирное начертание

3. StackPanel с кнопками:

```
<StackPanel Grid.Row="1" Orientation="Horizontal" Margin="10">
```

- Orientation="Horizontal" — кнопки располагаются горизонтально
- Каждая кнопка имеет фиксированную ширину и отступ справа

4. DataGrid с настроенными столбцами:

- Используется `DataGridTextBoxColumn` для текстовых полей
- Для столбца "Статус" используется `DataGridTemplateColumn` с конвертером для отображения на русском

Введение в ObservableCollection

Что такое ObservableCollection?

ObservableCollection — это специальная коллекция, которая автоматически уведомляет UI об изменениях (добавление, удаление, замена элементов).

Сравнение с обычным List:

```
// List<T> - НЕ уведомляет UI
List<Reservation> reservations = new();
reservations.Add(newReservation); // UI не обновится!

// ObservableCollection<T> - уведомляет UI
ObservableCollection<Reservation> reservations = new();
reservations.Add(newReservation); // UI автоматически обновится!
```

Когда использовать ObservableCollection?

- Для **ItemsSource** в **DataGrid**, **ListBox**, **ComboBox** — чтобы UI автоматически обновлялся при изменении коллекции
- Для **динамических списков** — когда элементы добавляются/удаляются во время работы приложения

Как работает ObservableCollection?

ObservableCollection реализует интерфейс **INotifyCollectionChanged**, который генерирует событие при изменении коллекции.

```
public ObservableCollection<Reservation> Reservations { get; set; } = new();

// При добавлении элемента:
Reservations.Add(newReservation);
// ObservableCollection генерирует событие CollectionChanged
// DataGrid подписан на это событие и автоматически обновляет отображение
```

Как ObservableCollection работает с репозиториями?

Важно понимать связь между репозиторием и ObservableCollection в UI:

Репозиторий возвращает List:

```
public List<Reservation> GetAll()
{
    return _reservations.Select(r => r.Clone()).ToList();
}
```

UI использует ObservableCollection:

```
public ObservableCollection<Reservation> Reservations { get; set; } = new();
```

Почему не просто присвоить?

```
// НЕПРАВИЛЬНО: DataGrid потеряет привязку
Reservations = new ObservableCollection<Reservation>(repository.GetAll());
```

Когда мы создаём новую ObservableCollection, старая коллекция, к которой был привязан DataGrid, остаётся в памяти, а DataGrid не узнаёт о новой коллекции.

Правильный способ — обновить существующую коллекцию:

```
private void LoadReservations()
{
    var reservations = _repository.GetAll(); // Получаем List<T> из репозитория
    Reservations.Clear();                    // Очищаем существующую ObservableCollection
    foreach (var reservation in reservations)
    {
        Reservations.Add(reservation);      // Добавляем элементы по одному
    }
}
```

Что происходит:

1. Reservations.Clear() — генерирует событие "коллекция очищена", DataGrid удаляет все строки
2. Каждый Reservations.Add() — генерирует событие "элемент добавлен", DataGrid добавляет новую строку
3. DataGrid остаётся привязанным к той же ObservableCollection

Когда вызывать LoadReservations():

- При загрузке окна (в конструкторе)
- После каждой операции CRUD:
 - После Create — чтобы новый элемент появился в списке
 - После Update — чтобы изменения отобразились
 - После Delete — чтобы элемент исчез из списка

Code-behind для формы списка

Создание code-behind файла

Файл: <ИмяПроекта>.UI/<Сущность>sListWindow.xaml.cs (например, ReservationsListWindow.xaml.cs)

```
namespace <ИмяПроекта>.UI;

using System.Collections.ObjectModel;
using System.Windows;
using <ИмяПроекта>.Data.Interfaces;
using <ИмяПроекта>.Domain;

public partial class <Сущность>sListWindow : Window
{
    private readonly I<Сущность>Repository _repository;
    private readonly I<Связанная>Repository _<связанная>Repository;

    public ObservableCollection<<Сущность>> Items { get; set; } = new();

    public <Сущность>? SelectedItem { get; set; }

    public <Сущность>sListWindow(I<Сущность>Repository repository, I<Связанная>Repository
<связанная>Repository)
```

```

{
    InitializeComponent();
    _repository = repository;
    _<связанная>Repository = <связанная>Repository;
    DataContext = this;
    LoadItems();
}

private void LoadItems()
{
    var items = _repository.GetAll();
    Items.Clear();
    foreach (var item in items)
    {
        Items.Add(item);
    }
}

private void CreateButton_Click(object sender, RoutedEventArgs e)
{
    var item = <Сущность>EditWindow.Create(_<связанная>Repository);
    if (item != null)
    {
        _repository.Add(item);
        LoadItems();
    }
}

private void EditButton_Click(object sender, RoutedEventArgs e)
{
    var selectedItem = SelectedItem;
    if (selectedItem is not null)
    {
        var editedItem = <Сущность>EditWindow.Edit(selectedItem, _<связанная>Repository);
        if (editedItem != null)
        {
            _repository.Update(editedItem);
            LoadItems();
        }
    }
}

private void DeleteButton_Click(object sender, RoutedEventArgs e)
{
    var selectedItem = SelectedItem;
    if (selectedItem is not null)
    {
        var result = MessageBox.Show(
            "Вы уверены, что хотите удалить эту запись?",
            "Подтверждение удаления",
            MessageBoxButton.YesNo,
            MessageBoxImage.Question);

        if (result == MessageBoxResult.Yes)
        {
            _repository.Delete(selectedItem);
            LoadItems();
        }
    }
}
}

```


Конкретный пример для резерваций:

Файл: BeautyDB.UI/ReservationsListWindow.xaml.cs

```
namespace BeautyDB.UI;

using System.Collections.ObjectModel;
using System.Windows;
using BeautyDB.Data.Interfaces;
using BeautyDB.Domain;

public partial class ReservationsListWindow : Window
{
    private readonly IReservationRepository _reservationRepository;
    private readonly IMasterRepository _masterRepository;

    public ObservableCollection<Reservation> Reservations { get; set; } = new();

    public Reservation? SelectedReservation { get; set; }

    public ReservationsListWindow(IReservationRepository reservationRepository, IMasterRepository masterRepository)
    {
        InitializeComponent();
        _reservationRepository = reservationRepository;
        _masterRepository = masterRepository;
        DataContext = this;
        LoadReservations();
    }

    private void LoadReservations()
    {
        var reservations = _reservationRepository.GetAll();
        Reservations.Clear();
        foreach (var reservation in reservations)
        {
            Reservations.Add(reservation);
        }
    }

    private void CreateButton_Click(object sender, RoutedEventArgs e)
    {
        var reservation = ReservationEditWindow.Create(_masterRepository);
        if (reservation != null)
        {
            _reservationRepository.Add(reservation);
            LoadReservations();
        }
    }

    private void EditButton_Click(object sender, RoutedEventArgs e)
    {
        var selectedReservation = SelectedReservation;
        if (selectedReservation is not null)
        {
            var editedReservation = ReservationEditWindow.Edit(selectedReservation, _masterRepository);
            if (editedReservation != null)
            {
                _reservationRepository.Update(editedReservation);
                LoadReservations();
            }
        }
    }
}
```

```

    }
}

private void DeleteButton_Click(object sender, RoutedEventArgs e)
{
    var selectedReservation = SelectedReservation;
    if (selectedReservation is not null)
    {
        var result = MessageBox.Show(
            "Вы уверены, что хотите удалить эту запись?",
            "Подтверждение удаления",
            MessageBoxButton.YesNo,
            MessageBoxImage.Question);

        if (result == MessageBoxResult.Yes)
        {
            _reservationRepository.Delete(selectedReservation);
            LoadReservations();
        }
    }
}
}

```

Разбор кода:

1. Конструктор с Dependency Injection:

```

public ReservationsListWindow(IReservationRepository reservationRepository, IMasterRepository
masterRepository)
{
    InitializeComponent();
    _reservationRepository = reservationRepository;
    _masterRepository = masterRepository;
    DataContext = this;
    LoadReservations();
}

```

- Репозитории передаются через конструктор (паттерн **Dependency Injection**)
- `DataContext = this` — окно само является контекстом данных для привязок

2. Метод LoadReservations():

```

private void LoadReservations()
{
    var reservations = _reservationRepository.GetAll();
    Reservations.Clear();
    foreach (var reservation in reservations)
    {
        Reservations.Add(reservation);
    }
}

```

- Получает данные из репозитория
- Очищает коллекцию `ObservableCollection`
- Заполняет коллекцию новыми данными
- **Важно:** Каждый раз вызываем после операций CRUD, чтобы обновить отображение

3. Обработчик CreateButton_Click:

```

var reservation = ReservationEditWindow.Create(_masterRepository);
if (reservation != null)
{
    _reservationRepository.Add(reservation);
}

```

```
LoadReservations();  
}
```

- Открывает окно создания через статический метод `Create()`, передавая ему `_masterRepository`
- Если пользователь сохранил (не отменил), добавляет в репозиторий
- Обновляет отображение вызовом `LoadReservations()`

4. Обработчик `EditButton_Click`:

```
var selectedReservation = SelectedReservation;  
if (selectedReservation is not null)  
{  
    var editedReservation = ReservationEditWindow.Edit(selectedReservation,  
_masterRepository);  
    if (editedReservation != null)  
    {  
        _reservationRepository.Update(editedReservation);  
        LoadReservations();  
    }  
}
```

- Проверяет, что строка выбрана
- Открывает окно редактирования через статический метод `Edit()`, передавая ему `_masterRepository`
- Если пользователь сохранил изменения, обновляет в репозитории
- Перезагружает список

5. Обработчик `DeleteButton_Click`:

```
var result = MessageBox.Show(  
    "Вы уверены, что хотите удалить эту запись?",  
    "Подтверждение удаления",  
    MessageBoxButton.YesNo,  
    MessageBoxImage.Question);  
  
if (result == MessageBoxResult.Yes)  
{  
    _reservationRepository.Delete(selectedReservation);  
    LoadReservations();  
}
```

- Показывает диалог подтверждения удаления
- `MessageBoxButton.YesNo` — кнопки "Да" и "Нет"
- `MessageBoxImage.Question` — иконка вопроса
- Удаляет из репозитория только если пользователь подтвердил

Почему нужна локальная переменная для `SelectedReservation`?

```
var selectedReservation = SelectedReservation;  
if (selectedReservation is not null)  
{  
    // Работаем с локальной переменной  
}
```

Причина: Свойство `SelectedReservation` может измениться в процессе выполнения метода (например, если пользователь кликнет на другую строку). Копирование в локальную переменную защищает от этой проблемы.

Настройка `App.xaml` для запуска формы списка

Изменение стартового окна

По умолчанию WPF запускает `MainWindow`. Нам нужно изменить это на наше новое окно списка.

Файл: `<ИмяПроекта>.UI/App.xaml`

Было:

```
<Application x:Class="<ИмяПроекта>.UI.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="MainWindow.xaml">
</Application>
```

Стало:

```
<Application x:Class="<ИмяПроекта>.UI.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
</Application>
```

Изменения:

- Удалили атрибут `StartupUri` (теперь будем запускать окно из кода)

Создание экземпляров репозиториев и запуск приложения

Файл: `<ИмяПроекта>.UI/App.xaml.cs`

```
namespace <ИмяПроекта>.UI;

using System.Windows;
using <ИмяПроекта>.Data.InMemory;
using <ИмяПроекта>.Data.Interfaces;

public partial class App : Application
{
    private I<Сущность>Repository _<сущность>Repository = null!;
    private I<Связанная>Repository? _<связанная>Repository;

    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);

        // Создаём экземпляры репозиториев
        _<сущность>Repository = new <Сущность>Repository();
        _<связанная>Repository = new <Связанная>Repository();

        // Заполняем тестовыми данными
        SeedInitialData();

        // Запускаем главное окно
        var mainWindow = new <Сущность>SListWindow(_<сущность>Repository);
        mainWindow.Show();
    }

    private void SeedInitialData()
    {
        // Добавьте тестовые данные здесь
    }
}
```

Конкретный пример для салона красоты:

Файл: BeautyDB.UI/App.xaml.cs

```
namespace BeautyDB.UI;

using System.Windows;
using BeautyDB.Data.InMemory;
using BeautyDB.Data.Interfaces;
using BeautyDB.Domain;

public partial class App : Application
{
    private IReservationRepository _reservationRepository = null!;
    private IMasterRepository _masterRepository = null!;

    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);

        // Создаём экземпляры репозиториев
        _reservationRepository = new ReservationRepository();
        _masterRepository = new MasterRepository();

        // Заполняем тестовыми данными
        SeedInitialData();

        // Запускаем главное окно
        var mainWindow = new ReservationsListWindow(_reservationRepository, _masterRepository);
        mainWindow.Show();
    }

    private void SeedInitialData()
    {
        // Создаём тестовых мастеров
        var masters = new List<Master>
        {
            new Master { Id = Guid.NewGuid(), Name = "Анна Иванова" },
            new Master { Id = Guid.NewGuid(), Name = "Мария Петрова" },
            new Master { Id = Guid.NewGuid(), Name = "Елена Сидорова" }
        };

        _masterRepository.Seed(masters);

        // Создаём тестовые резервации
        _reservationRepository.Add(new Reservation
        {
            Id = Guid.Empty, // Будет заменён в репозитории
            Client = new Client { Name = "Иван Иванов", Phone = "+7 (900) 123-45-67" },
            Master = masters[0],
            ServiceDescription = "Стрижка и укладка",
            Date = DateOnly.FromDateTime(DateTime.Today.AddDays(1)),
            StartTime = new TimeOnly(10, 0),
            Duration = TimeSpan.FromMinutes(60),
            Status = ReservationStatus.Confirmed
        });

        _reservationRepository.Add(new Reservation
        {
            Id = Guid.Empty,
            Client = new Client { Name = "Мария Петрова", Phone = "+7 (900) 234-56-78" },
            Master = masters[1],
```

```

        ServiceDescription = "Окрашивание волос",
        Date = DateOnly.FromDateTime(DateTime.Today.AddDays(2)),
        StartTime = new TimeOnly(14, 0),
        Duration = TimeSpan.FromHours(2),
        Status = ReservationStatus.Created
    });

    _reservationRepository.Add(new Reservation
    {
        Id = Guid.Empty,
        Client = new Client { Name = "Анна Сидорова", Phone = "+7 (900) 345-67-89" },
        Master = masters[2],
        ServiceDescription = "Маникюр",
        Date = DateOnly.FromDateTime(DateTime.Today),
        StartTime = new TimeOnly(16, 30),
        Duration = TimeSpan.FromMinutes(45),
        Status = ReservationStatus.InProgress
    });
}
}

```

Разбор кода:

1. Поля для репозиториев:

```
private IReservationRepository _reservationRepository = null!;
```

- `null!` — оператор прощения `null` (говорит компилятору, что мы знаем, что инициализируем поле позже)

2. Метод `OnStartup`:

```
protected override void OnStartup(StartupEventArgs e)
```

- Вызывается при запуске приложения (до показа окна)
- `base.OnStartup(e)` — вызываем базовую реализацию

3. Создание репозиториев:

```
_reservationRepository = new ReservationRepository();
_masterRepository = new MasterRepository();
```

- Создаём конкретные реализации репозиториев
- Храним их в полях типа интерфейсов (следуем принципу **Dependency Inversion**)

4. Метод `SeedInitialData()`:

- Заполняет репозитории начальными и тестовыми данными
- Вызывается один раз при запуске приложения
- Без этого список будет пустым при первом запуске
- Тут заполняем списки вспомогательных сущностей, которые никак иначе заполнить нельзя

5. Запуск главного окна:

```
var mainWindow = new ReservationsListWindow(_reservationRepository, _masterRepository);
mainWindow.Show();
```

- Создаём окно, передавая ему репозиторий через конструктор
- `Show()` — показываем окно (неблокирующий вызов)

Удаление `MainWindow` (опционально)

Если вы больше не используете `MainWindow`, можете его удалить:

1. Удалите файлы `MainWindow.xaml` и `MainWindow.xaml.cs`

Обновление ReservationEditWindow для работы с репозиториями

Зачем нужно обновление?

В предыдущей работе мы использовали статический список мастеров в `ReservationEditWindow`. Теперь мастера хранятся в репозитории, и нам нужно загружать их оттуда.

Изменение конструктора ReservationEditWindow

Было:

```
public static ObservableCollection<Master> AvailableMasters { get; set; } = new()
{
    new Master { Id = Guid.NewGuid(), Name = "Анна Иванова" },
    new Master { Id = Guid.NewGuid(), Name = "Мария Петрова" },
    new Master { Id = Guid.NewGuid(), Name = "Елена Сидорова" }
};
```

Стало:

```
private readonly IMasterRepository _masterRepository;

public ObservableCollection<Master> AvailableMasters { get; set; } = new();

private ReservationEditWindow(Reservation? reservation, IMasterRepository masterRepository)
{
    InitializeComponent();
    _masterRepository = masterRepository;

    // Загружаем мастеров из репозитория
    var masters = _masterRepository.GetAll();
    AvailableMasters = new ObservableCollection<Master>(masters);

    _reservation = reservation is null ? Reservation.Create() : reservation.Clone();
    DataContext = this;
}
```

Обновление статических методов Create и Edit

Файл: `BeautyDB.UI/ReservationEditWindow.xaml.cs` (фрагмент)

```
public static Reservation? Create(IMasterRepository masterRepository)
{
    var window = new ReservationEditWindow(null, masterRepository);
    window.Title = "Создание резервации";
    return window.ShowDialog() == true ? window._reservation : null;
}

public static Reservation? Edit(Reservation reservation, IMasterRepository masterRepository)
{
    var window = new ReservationEditWindow(reservation, masterRepository);
    window.Title = "Редактирование резервации";
    return window.ShowDialog() == true ? window._reservation : null;
}
```

Обновление вызовов в ReservationsListWindow

Было:

```
var reservation = ReservationEditWindow.Create();
```

Стало:

```
var reservation = ReservationEditWindow.Create(_masterRepository);
```

Полный код обработчиков:

```
private void CreateButton_Click(object sender, RoutedEventArgs e)
{
    var reservation = ReservationEditWindow.Create(_masterRepository);
    if (reservation != null)
    {
        _reservationRepository.Add(reservation);
        LoadReservations();
    }
}

private void EditButton_Click(object sender, RoutedEventArgs e)
{
    var selectedReservation = SelectedReservation;
    if (selectedReservation is not null)
    {
        var editedReservation = ReservationEditWindow.Edit(selectedReservation,
            _masterRepository);
        if (editedReservation != null)
        {
            _reservationRepository.Update(editedReservation);
            LoadReservations();
        }
    }
}
```

Фиксация изменений в Git

Проверка изменений

Перед созданием коммита проверьте, какие файлы были изменены:

```
git status
```

Вы должны увидеть:

- Новые проекты `Data.Interfaces` и `Data.InMemory`
- Новые файлы репозитория
- Новое окно `ReservationsListWindow`
- Изменения в `App.xaml` и `App.xaml.cs`
- Изменения в `ReservationEditWindow`

Просмотр изменений

Чтобы увидеть конкретные изменения в файлах:

```
git diff
```


Добавление файлов в коммит

Добавьте все изменённые файлы:

```
git add .
```

Альтернатива: Можно добавлять файлы выборочно:

```
git add BeautyDB.Data.Interfaces/  
git add BeautyDB.Data.InMemory/  
git add BeautyDB.UI/ReservationsListWindow.xaml  
# и т.д.
```

Создание коммита

Создайте коммит с описанием выполненной работы:

```
git commit -m "Добавление репозитория и формы списка"
```

Просмотр истории

Проверьте, что коммит создан:

```
git log --oneline
```

Тестирование приложения

Проверка функциональности

Запустите приложение и протестируйте следующие сценарии:

1. **Отображение списка:**

- ☐ При запуске отображаются тестовые данные
- ☐ Все столбцы отображаются корректно
- ☐ Прокрутка работает, если данных много

2. **Создание записи:**

- ☐ Кнопка "Создать" открывает окно редактирования
- ☐ Заполнение всех полей и сохранение добавляет запись в список
- ☐ Новая запись появляется в DataGrid
- ☐ Кнопка "Отмена" закрывает окно без добавления

3. **Редактирование записи:**

- ☐ Выбор строки и нажатие "Редактировать" открывает окно с данными
- ☐ Изменение полей и сохранение обновляет запись в списке
- ☐ Кнопка "Отмена" закрывает окно без изменений

4. **Удаление записи:**

- ☐ Выбор строки и нажатие "Удалить" показывает диалог подтверждения
- ☐ Подтверждение удаляет запись из списка
- ☐ Отмена оставляет запись в списке

5. **Валидация:**

- ☐ Попытка сохранить запись с пустыми обязательными полями показывает ошибку
- ☐ Некорректный формат времени показывает ошибку

Типичные проблемы и их решение

Проблема: DataGrid пустой, хотя данные есть в репозитории

Решение:

- Проверьте, что `ItemsSource="{Binding Reservations}"` в XAML
 - Проверьте, что `DataContext = this;` установлен в конструкторе окна
 - Проверьте, что свойство `Reservations` публичное
-

Проблема: Изменения в столбцах DataGrid не отображаются после редактирования

Решение:

- Убедитесь, что вызываете `LoadReservations()` после `Update()`
 - Проверьте, что в репозитории используется `Clone()` для защиты данных
-

Проблема: Кнопка "Редактировать" не открывает окно

Решение:

- Проверьте, что `SelectedItem="{Binding SelectedReservation}"` установлен в DataGrid
 - Проверьте, что свойство `SelectedReservation` публичное
-

Проблема: ComboBox с мастерами пустой в форме редактирования

Решение:

- Проверьте, что передаёте `IMasterRepository` в конструктор `ReservationEditWindow`
 - Убедитесь, что метод `_masterRepository.GetAll()` возвращает данные
 - Проверьте, что в `App.xaml.cs` вызван `SeedInitialData()` перед созданием окна
-

Заключение

В данной работе мы изучили:

1. **Паттерн Repository** — как разделить логику хранения данных от UI
2. **Интерфейсы и их реализации** — создание контрактов и конкретных реализаций
3. **In-memory хранилище** — использование `List<T>` для временного хранения данных
4. **DataGrid** — отображение табличных данных в WPF
5. **ObservableCollection** — автоматическое обновление UI при изменении коллекции
6. **CRUD операции** — создание, чтение, обновление и удаление данных
7. **Dependency Injection** — передача зависимостей через конструктор
8. **Работу с Git** — фиксация изменений и создание описательных коммитов

Дополнительные ресурсы

- Паттерн Repository: <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design>
- Dependency Injection: <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>

- Официальная документация WPF DataGrid: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/controls/datagrid>
- ObservableCollection: <https://learn.microsoft.com/en-us/dotnet/api/system.collections.objectmodel.observablecollection-1>
- LINQ: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/>